

Module 2 — Core Skills Drill: Pipeline Basics

When this is assigned: Available after the Key Concept Session (Day 1 evening). Due the same night.

Time estimate: 15–20 minutes.

Submission: [Accept the M2-D2-pipeline-basics assignment](#), then open a PR from branch `drill-2-pipeline-basics` to `main`. Paste the PR URL into the TalentLMS text submission field for Unit 4.

What This Drill Practices

The reading and concept session introduced pandas data functions and pytest testing. This drill moves from recognition to recall: you write these from memory, without looking up syntax, using only your understanding of the concepts.

Scope: Data pipeline functions + pytest. No PyTorch — that retrieval practice comes through the integration task later in the week.

Setup

In your `M2-D2-pipeline-basics` assignment repo ([Accept the assignment](#) if you haven't already), create a new branch:

```
git checkout -b drill-2-pipeline-basics
```

You will create two files in the repo root:

- `drill_functions.py` — your data functions
- `test_drill.py` — your pytest tests

Task 1 — Write `clean_column`

In `drill_functions.py`, write a function `clean_column(series)` that:

- Takes a pandas `Series` as its argument
- Fills any `NaN` values with the series **median**
- Returns the cleaned `Series`

```
# drill_functions.py

import pandas as pd

def clean_column(series):
    """Fill NaN values with the series median. Returns the cleaned Series."""
    # TODO: implement this function
    pass
```

Task 2 — Write `compute_revenue`

In the same `drill_functions.py`, write a function `compute_revenue(quantity, price)` that:

- Takes two pandas `Series` as arguments (`quantity` and `unit_price`)
- Multiplies them element-wise
- Returns the resulting revenue `Series`

```
def compute_revenue(quantity, price):
    """Multiply quantity and price element-wise. Returns a revenue Series."""
    # TODO: implement this function
    pass
```

Task 3 — Write Tests

In `test_drill.py`, write **two** pytest test functions:

```
test_clean_column() :

    • Create a pd.Series with at least one NaN value
    • Call clean_column() on it
    • Assert that no NaN values remain in the result
    • Assert that the NaN was filled with the correct median value

test_compute_revenue() :

    • Create two small pd.Series (quantity and price)
    • Call compute_revenue() on them
    • Assert that the result matches the expected element-wise product

# test_drill.py

import pandas as pd
import numpy as np
from drill_functions import clean_column, compute_revenue

def test_clean_column() :
    # TODO: create a Series with at least one NaN, call clean_column(),
    # assert no NaN remain, assert the fill value is correct
    pass

def test_compute_revenue() :
    # TODO: create two small Series, call compute_revenue(),
    # assert the result matches the expected element-wise product
    pass
```

Task 4 — Run and Confirm

Run your tests locally:

```
pytest test_drill.py -v
```

Both tests must pass before you submit. If a test fails, fix your function (or your test) until it passes.

Submit

- Stage and commit your two files:

```
git add drill_functions.py test_drill.py
git commit -m "Add drill-2-pipeline-basics: clean_column, compute_revenue, tests"
```

- Push your branch:

```
git push -u origin drill-2-pipeline-basics
```

- Open a PR from `drill-2-pipeline-basics` to `main` in your assignment repo.
- The autograder runs automatically. Wait for the GitHub Actions check to complete — it will show a green checkmark (pass) or red X (fail) next to your latest commit.
- Once the autograder passes, paste the PR URL into the TalentLMS Unit 4 text field.

What the Autograder Checks

The CI autograder verifies:

- `drill_functions.py` exists
- `clean_column()` fills `NaN` with the correct median value
- `compute_revenue()` multiplies two `Series` element-wise correctly
- `test_drill.py` exists
- Your own tests in `test_drill.py` pass

If the autograder fails, check the Actions tab on your PR for the error output. Fix the issue, commit, and push — the autograder re-runs automatically.

